AI PRODUCT MANAGEMENT 2 · DETAILED COURSE

# Agile, Scrum and Product Delivery

How teams plan, build and ship iteratively, and the tools PMs use to coordinate delivery.

Chapter 06 of 26 · 23 topics · Edition 2026 · skillnavora.in

CHAPTER 06

# Agile, Scrum and Product Delivery

How teams plan, build and ship iteratively, and the tools PMs use to coordinate delivery.

## Agile Fundamentals

**Iterative delivery, frequent feedback and adapting to change over following a fixed plan.**

Agile delivers work in short iterations, gathers feedback quickly and adapts plans to what is learned. It replaces long fixed plans with frequent checkpoints. The PM's job is to keep the backlog ordered by outcome value.

## Agile Manifesto

**Four values and twelve principles favouring individuals, working software, collaboration and responding to change.**

The manifesto values individuals and interactions, working software, customer collaboration and responding to change. The twelve principles add practical guidance such as frequent delivery and sustainable pace. Use the values as a test for practices, not as a rulebook.

## Scrum Framework

**Fixed-length sprints with defined roles, events and artefacts to deliver increments.**

Scrum organises work into fixed-length sprints, with defined roles, events and artefacts. Each sprint produces an increment of working product. Teams adopt Scrum to create a predictable rhythm of planning, review and improvement.

## Kanban Methodology

**A continuous flow board that limits work in progress and optimises throughput.**

Kanban visualises work on a board, limits work in progress and measures flow. It suits support and operations work with unpredictable arrivals. Limits prevent teams from starting too much and finishing nothing.

## Scrum Roles and Responsibilities

**Product Owner, Scrum Master and Developers, each accountable for a distinct part of the work.**

The product owner orders the backlog and maximises value. The Scrum master removes impediments and coaches the process. Developers build the increment. Each role has a distinct accountability, and blurring them is a common failure.

## Product Owner Responsibilities

**Maximising product value by managing and ordering the backlog and accepting work.**

The product owner manages the backlog, writes clear items, accepts work against criteria and represents customers to the team. A product owner who is absent from daily decisions cannot steer the team well.

## Sprint Planning

**The team selects backlog items and plans how to deliver them within the sprint goal.**

The team selects backlog items and agrees a sprint goal. Planning answers what can be delivered and how. A good sprint goal describes the outcome, so the team can adapt scope without losing purpose.

## Daily Stand-ups

**A 15-minute sync on progress toward the sprint goal and blockers.**

The daily stand-up is a 15-minute check on progress toward the sprint goal and on blockers. It is not a status report to the manager. Use it to remove blockers quickly, for example by escalating a waiting API decision the same day.

## Sprint Reviews

**Demonstrating the increment to stakeholders and gathering feedback.**

The sprint review demonstrates the working increment to stakeholders and gathers feedback. It is the best place to show real software, not slides. Feedback here should change the backlog order.

## Sprint Retrospectives

**The team reflects on how it worked and agrees small improvements.**

The retrospective examines how the team worked and agrees small, specific improvements. Keep one or two actions per sprint and track whether they helped. Retrospectives fail when they produce long lists nobody follows.

## Backlog Grooming

**Older term for refinement: keeping the backlog clear, sized and ordered.**

Backlog grooming is the older term for refinement: keeping items clear, sized and ordered. Groom continuously rather than in one large session, so the team is never starting from scratch.

## Story Points

**A relative unit of effort and complexity for sizing stories, not hours.**

Story points measure relative size and complexity, not hours. A story of 5 points should feel about five times the effort of a 1-point story. Use points for forecasting, and do not compare teams by points.

## Estimation Techniques

**Planning poker, t-shirt sizing, affinity estimation and reference-class estimates to forecast work.**

Planning poker, t-shirt sizing and affinity estimation provide quick estimates. Reference-class estimates compare work to similar past items. Estimates are forecasts with error bars, so communicate ranges.

## Velocity and Burndown Charts

**Velocity is points completed per sprint; burndown shows remaining work over time. Use for forecasting, not for judging people.**

Velocity is the points completed per sprint, and burndown shows remaining work over time. They forecast delivery. Using them to judge individuals damages trust and distorts estimates.

## Release Planning

**Sequencing sprints into releases with dates, scope, dependencies and risk buffers; staged rollout and rollback plans belong here.**

Release planning sequences sprints into releases with scope, dates, dependencies and risk buffers. Staged rollouts such as 5%, 25% and 100% of users, with rollback plans, belong here. A release is ready when its go/no-go criteria are met.

## Dependency Management

**Identifying and tracking work that relies on other teams, systems or vendors, and resolving early.**

A dependency is work that relies on another team, system or vendor. List dependencies early, agree dates with those teams and track them in each sprint review. Hidden dependencies cause most missed release dates.

## Risk and Issue Management

**Risks are possible problems to mitigate; issues are problems that have occurred and need action.**

A risk is a possible problem you can prevent or reduce. An issue is a problem that has already happened and needs action. Keep a short risk register with owners, likelihood, impact and the mitigation date.

## Cross-Functional Team Collaboration

**Product, design, engineering, data, QA and go-to-market working toward one goal with shared context.**

Product, design, engineering, data, QA and go-to-market work toward one goal with shared context. Shared goals and regular demos prevent the handoff delays of separate silos.

## Engineering Collaboration

**Working with engineers on trade-offs, feasibility and scope, treating them as partners from discovery onward.**

Involve engineers from discovery onward. They know the cost of options and can suggest simpler solutions. Treat trade-offs on scope and quality as joint decisions.

## Product Delivery Lifecycle

**Idea to discovery, design, build, test, release, measure and iterate.**

The lifecycle runs from idea to discovery, design, build, test, release, measurement and iteration. Measurement closes the loop: the results decide whether to invest more, change direction or stop.

### Jira

**Issue and sprint tracking tool for backlogs, boards, workflows and reports.**

Jira tracks issues, sprints, boards and workflows. Keep the workflow simple, with few statuses, and use a clear naming and labelling convention. Reports are useful only if the data is kept current.

### Confluence

**Team wiki used for specs, decision records and knowledge sharing, integrated with Jira.**

Confluence is a team wiki for specifications, decision records and knowledge. Link pages to Jira issues so context stays connected to the work. Archive old pages so people can find current ones.

### Linear and Azure DevOps

**Linear is a fast, opinionated issue tracker; Azure DevOps combines boards, repos and pipelines for engineering teams.**

Linear is a fast, opinionated issue tracker popular with product teams. Azure DevOps combines boards, repositories and pipelines for engineering organisations. Choose the tool that fits how your engineering team already works.

## Summary of terms

| Term | Meaning |
|---|---|
| **Agile Fundamentals** | Iterative delivery, frequent feedback and adapting to change over following a fixed plan. |
| **Agile Manifesto** | Four values and twelve principles favouring individuals, working software, collaboration and responding to change. |
| **Scrum Framework** | Fixed-length sprints with defined roles, events and artefacts to deliver increments. |
| **Kanban Methodology** | A continuous flow board that limits work in progress and optimises throughput. |
| **Scrum Roles and Responsibilities** | Product Owner, Scrum Master and Developers, each accountable for a distinct part of the work. |
| **Product Owner Responsibilities** | Maximising product value by managing and ordering the backlog and accepting work. |
| **Sprint Planning** | The team selects backlog items and plans how to deliver them within the sprint goal. |
| **Daily Stand-ups** | A 15-minute sync on progress toward the sprint goal and blockers. |
| **Sprint Reviews** | Demonstrating the increment to stakeholders and gathering feedback. |
| **Sprint Retrospectives** | The team reflects on how it worked and agrees small improvements. |
| **Backlog Grooming** | Older term for refinement: keeping the backlog clear, sized and ordered. |
| **Story Points** | A relative unit of effort and complexity for sizing stories, not hours. |
| **Estimation Techniques** | Planning poker, t-shirt sizing, affinity estimation and reference-class estimates to forecast work. |
| **Velocity and Burndown Charts** | Velocity is points completed per sprint; burndown shows remaining work over time. Use for forecasting, not for judging people. |

| | |
|---|---|
| **Release Planning** | Sequencing sprints into releases with dates, scope, dependencies and risk buffers; staged rollout and rollback plans belong here. |
| **Dependency Management** | Identifying and tracking work that relies on other teams, systems or vendors, and resolving early. |
| **Risk and Issue Management** | Risks are possible problems to mitigate; issues are problems that have occurred and need action. |
| **Cross-Functional Team Collaboration** | Product, design, engineering, data, QA and go-to-market working toward one goal with shared context. |
| **Engineering Collaboration** | Working with engineers on trade-offs, feasibility and scope, treating them as partners from discovery onward. |
| **Product Delivery Lifecycle** | Idea to discovery, design, build, test, release, measure and iterate. |
| **Jira** | Issue and sprint tracking tool for backlogs, boards, workflows and reports. |
| **Confluence** | Team wiki used for specs, decision records and knowledge sharing, integrated with Jira. |
| **Linear and Azure DevOps** | Linear is a fast, opinionated issue tracker; Azure DevOps combines boards, repos and pipelines for engineering teams. |

**Practice exercise:** Break one epic into stories small enough for a sprint, estimate them with planning poker and draft the sprint goal.

**Self-check**

1. Explain Agile Fundamentals in two sentences to a non-technical stakeholder, then say what you would measure for it.
2. Explain Story Points in two sentences to a non-technical stakeholder, then say what you would measure for it.
3. Explain Linear and Azure DevOps in two sentences to a non-technical stakeholder, then say what you would measure for it.

## Related SkillNavora material

- AI Tutor: Ask questions on any topic.
- Study Library: Every lesson, class and PDF by programme.

© SkillNavora · skillnavora.in · Facts and tools change; check official sites before relying on a figure.